

Youtube link to unlisted video of Milestone 1: <https://youtu.be/y6RHhPgsoul>

Youtube link to unlisted video of Milestone 2: <https://youtu.be/l1Wme0xCTYo>

Youtube link to unlisted video of Milestone 3: <https://youtu.be/9BarOUTuxx8>

From the videos, we can see the car passes all requirements for a grade 5:

"Grade 5: Car behaves properly (car drives using regular rules, does not tilt from side to side, can stop where it needs to and moves at a reasonable pace)"

Here we provide the modified .py -files. ai_test.py is included because we slightly adjusted the waypoints for the tests. The exact changes are mentioned in the comments.

Both team members contributed equally. We have worked together during calls.

ai_control.py

```
#!/usr/bin/env python

import glob
import os
import sys
from collections import deque
import math
import numpy as np

try:
    sys.path.append(glob.glob('*/**/*%d.%d-%s.egg' % (
        sys.version_info.major,
        sys.version_info.minor,
        'win-amd64' if os.name == 'nt' else 'linux-x86_64'))[0])
except IndexError:
    pass

import carla
import ai_knowledge as data
from ai_knowledge import Status

# Import CARLA's GlobalRoutePlanner (GRP)
try:
    from agents.navigation.global_route_planner import GlobalRoutePlanner
    HAS_GRP = True
except ImportError:
    HAS_GRP = False

# Distance threshold beyond which GRP is used
GRP_DISTANCE_THRESHOLD = 50.0

# GRP deactivates when this many meters is left of route
GRP_HANDOFF_DISTANCE = 20.0

# Executor is responsible for moving the vehicle around
```

```

# In this implementation it only needs to match the steering and speed so that we arrive
at provided waypoints
# BONUS TODO: implement different speed limits so that planner would also provide speed
target speed in addition to direction
class Executor(object):
    def __init__(self, knowledge, vehicle):
        self.vehicle = vehicle
        self.knowledge = knowledge
        self.target_pos = knowledge.get_location()

    # Update the executor at some intervals to steer the car in desired direction
    def update(self, time_elapsed):
        status = self.knowledge.get_status()
        # TODO: this needs to be able to handle
        if status == Status.DRIVING:
            dest = self.knowledge.get_current_destination()
            self.update_control(dest, [1], time_elapsed)
        elif status == Status.HEALING:
            self.update_heal(time_elapsed)
        elif status == Status.ARRIVED:
            self.update_stop()

        # TODO: steer in the direction of destination and throttle or brake depending on how
        close we are to destination
        # TODO: Take into account that exiting the crash site could also be done in reverse,
        so there might need to be additional data passed between planner and executor,
        # or there needs to be some way to tell this that it is ok to drive in reverse during
        HEALING and CRASHED states. An example is additional_vars,
        # that could be a list with parameters that can tell us which things we can do (for
        example going in reverse)
    def update_control(self, destination, additional_vars, delta_time):
        # Assign current vehicle state into variables, location and rotation
        current_loc = self.knowledge.get_location()
        current_rot = self.knowledge.retrieve_data('rotation')

        # Read target_speed (km/h) from knowledge and convert to m/s
        # Default speed is 8 m/s
        if 'target_speed' in self.knowledge.memory:
            target_speed = self.knowledge.retrieve_data('target_speed') / 3.6
        else:
            target_speed = 8.0

        # Required target speed can be passed via additional_vars parameter
        if len(additional_vars) > 1 and additional_vars[1] is not None:
            target_speed = additional_vars[1]

        # Compute vector from current position to destination
        dx = destination.x - current_loc.x
        dy = destination.y - current_loc.y
        dist = math.sqrt(dx * dx + dy * dy)

        # Compute forward vector of the vehicle from its yaw angle
        yaw_rad = math.radians(current_rot.yaw)

```

```

forward = np.array([math.cos(yaw_rad), math.sin(yaw_rad)])

# Compute normalized direction vector to destination
if dist < 0.001:
    direction = forward.copy()
else:
    direction = np.array([dx / dist, dy / dist])

# Calculate steering angle using dot and cross products
dot = np.clip(np.dot(forward, direction), -1.0, 1.0)
angle = math.acos(dot)

# cross product (z-component) determines left (-) or right (+) turn
cross = forward[0] * direction[1] - forward[1] * direction[0]
if cross < 0:
    angle = -angle

# Normalize steering to -1, 1. Max steer assumed at pi/2 or 90 degrees
steer = angle / (math.pi / 2.0)
steer = max(-1.0, min(1.0, steer))

# additional_vars[2]: destination rotation so vehicle arrives at correct angle
if len(additional_vars) > 2 and additional_vars[2] is not None:
    dest_rotation = additional_vars[2]
    # Blend toward the target arrival heading when close to destination
    if dist < 10.0:
        target_yaw_rad = math.radians(dest_rotation.yaw)
        target_forward = np.array([math.cos(target_yaw_rad), math.sin(target_yaw_rad)])
        dot2 = np.clip(np.dot(forward, target_forward), -1.0, 1.0)
        angle2 = math.acos(dot2)
        cross2 = forward[0] * target_forward[1] - forward[1] * target_forward[0]
        if cross2 < 0:
            angle2 = -angle2
        # Weight arrival correction more heavily as we get closer
        blend = 1.0 - (dist / 10.0)
        steer = (1.0 - blend) * steer + blend * (angle2 / (math.pi / 2.0))
        steer = max(-1.0, min(1.0, steer))

# Throttle and brake based on current speed vs target speed
current_vel = self.vehicle.get_velocity()
current_speed = math.sqrt(
    current_vel.x ** 2 + current_vel.y ** 2 + current_vel.z ** 2
)

# Apply end-of-route slowdown only while still on GRP waypoints and only
# when close to the end of the GRP chain
waypoints_remaining = self.knowledge.retrieve_data('waypoints_remaining') \
    if 'waypoints_remaining' in self.knowledge.memory else 999
using_grp = self.knowledge.retrieve_data('using_grp') \
    if 'using_grp' in self.knowledge.memory else False
on_final_direct_leg = self.knowledge.retrieve_data('on_final_direct_leg') \
    if 'on_final_direct_leg' in self.knowledge.memory else False

```

```

speed_factor = 1.0
if using_grp and not on_final_direct_leg and waypoints_remaining <= 5:
    # Slow down linearly over the last 5 GRP waypoints
    speed_factor = max(0.2, waypoints_remaining / 5.0)

adjusted_target = target_speed * speed_factor
speed_diff = adjusted_target - current_speed

# Speed settings
if target_speed <= 0:
    throttle = 0.0
    brake = 1.0
elif speed_diff > 0:
    throttle = min(1.0, speed_diff / target_speed)
    brake = 0.0
else:
    throttle = 0.0
    brake = min(1.0, -speed_diff / target_speed)

# Apply control
control = carla.VehicleControl()
control.throttle = throttle
control.steer = steer
control.brake = brake
control.hand_brake = False
self.vehicle.apply_control(control)

# HEALING behaviour
def update_heal(self, delta_time):
    control = carla.VehicleControl()
    control.throttle = 0.0
    control.steer = 0.0
    control.brake = 1.0
    control.hand_brake = False
    self.vehicle.apply_control(control)

# Stop the vehicle once the final destination is reached
def update_stop(self):
    control = carla.VehicleControl()
    control.throttle = 0.0
    control.steer = 0.0
    control.brake = 1.0
    self.vehicle.apply_control(control)

# Planner is responsible for creating a plan for moving around
# In our case it creates a list of waypoints to follow so that vehicle arrives at
destination
# Alternatively this can also provide a list of waypoints to try avoid crashing or
'uncrash' itself
class Planner(object):
    def __init__(self, knowledge):
        self.knowledge = knowledge

```

```

self.path = deque([])

# Create a map of waypoints to follow to the destination and save it
def make_plan(self, source, destination):
    self.path = self.build_path(source, destination)
    if isinstance(destination, carla.Transform):
        self.knowledge.destination = destination.location
    elif isinstance(destination, carla.Vector3D):
        self.knowledge.destination = carla.Location(destination)
    elif hasattr(destination, 'transform'):
        self.knowledge.destination = destination.transform.location
    else:
        self.knowledge.destination = destination
    self.update_plan()
    self.knowledge.update_destination(self.get_current_destination())

# Function that is called at time intervals to update ai-state
def update(self, time_elapsed):
    self.update_plan()
    self.knowledge.update_destination(self.get_current_destination())

# Update internal state to make sure that there are waypoints to follow and that we
have not arrived yet
def update_plan(self):
    if len(self.path) == 0:
        self.knowledge.update_data('waypoints_remaining', 0)
        return

    current_status = self.knowledge.get_status()
    if current_status not in (Status.DRIVING, Status.ARRIVED):
        return

    if self.knowledge.arrived_at(self.path[0]):
        self.path.popleft()

    if len(self.path) == 0:
        self.knowledge.update_status(Status.ARRIVED)
    else:
        self.knowledge.update_status(Status.DRIVING)

# Publish remaining waypoint count so executor can apply end-of-route braking
without relying on knowledge.destination
self.knowledge.update_data('waypoints_remaining', len(self.path))

# Signal to executor when we have consumed all GRP waypoints and only the final
direct destination waypoint remains
on_final = self.knowledge.retrieve_data('grp_waypoint_count') is not None and
len(self.path) <= 1
self.knowledge.update_data('on_final_direct_leg', on_final)

# Get current destination
def get_current_destination(self):
    status = self.knowledge.get_status()

```

```

# if we are driving, then the current destination is next waypoint
if status == Status.DRIVING:
    # TODO: Take into account traffic lights and other cars
    return self.path[0]
if status == Status.ARRIVED:
    return self.knowledge.get_location()
if status == Status.HEALING:
    # TODO: Implement crash handling. Probably needs to be done by following waypoint
list to exit the crash site.
    # Afterwards needs to remake the path.
    return self.knowledge.get_location()
if status == Status.CRASHED:
    # TODO: implement function for crash handling, should provide map of waypoints to
move towards to for exiting crash state.
    # You should use separate waypoint list for that, to not mess with the original
path.
    return self.knowledge.get_location()
# otherwise destination is same as current position
return self.knowledge.get_location()

# Create path of waypoints from source to destination
# Test 1: short route, no GRP needed
# Tests 2 & 3: long route, GRP waypoints up to GRP_HANDOFF_DISTANCE meters from the
destination,
# then a single direct waypoint for the final stretch
def build_path(self, source, destination):
    self.path = deque([])

    # Normalise source and destination to carla.Location
    if isinstance(source, carla.Transform):
        src_loc = source.location
    else:
        src_loc = source

    if isinstance(destination, carla.Transform):
        dest_loc = destination.location
    elif isinstance(destination, carla.Vector3D):
        dest_loc = carla.Location(destination)
    elif hasattr(destination, 'transform'):
        dest_loc = destination.transform.location
    else:
        dest_loc = destination

    straight_dist = src_loc.distance(dest_loc)

    # Test 1, simple single-waypoint path
    if straight_dist < GRP_DISTANCE_THRESHOLD or not HAS_GRP \
        or 'world' not in self.knowledge.memory:
        self.knowledge.update_data('using_grp', False)
        self.knowledge.update_data('on_final_direct_leg', False)
        self.knowledge.update_data('grp_waypoint_count', None)
        self.path.append(dest_loc)
    return self.path

```

```

# Tests 2 & 3, GRP + single-waypoint path
self.knowledge.update_data('using_grp', True)
self.knowledge.update_data('on_final_direct_leg', False)

world      = self.knowledge.retrieve_data('world')
carla_map = world.get_map()

try:
    grp = GlobalRoutePlanner(carla_map, sampling_resolution=2.0)
    route = grp.trace_route(src_loc, dest_loc)

    # Add remaining GRP waypoints after grp_handoff to variable
    grp_count = 0
    for wp, _ in route:
        loc = wp.transform.location
        if loc.distance(dest_loc) > GRP_HANDOFF_DISTANCE:
            self.path.append(loc)
            grp_count += 1

    # This way update_plan() can detect when only the final waypoint remains
    self.knowledge.update_data('grp_waypoint_count', grp_count)

    # Final waypoint left, handled like test 1
    self.path.append(dest_loc)
    return self.path
except Exception as e:
    print(f'[Planner] GlobalRoutePlanner failed ({e}), falling back to direct path')
    self.knowledge.update_data('using_grp', False)
    self.knowledge.update_data('on_final_direct_leg', False)
    self.knowledge.update_data('grp_waypoint_count', None)
    self.path = deque([dest_loc])
    return self.path

```

ai_parser.py

```

#!/usr/bin/env python

import glob
import os
import sys

try:
    sys.path.append(glob.glob('*/**/*d.%d-%s.egg' % (
        sys.version_info.major,
        sys.version_info.minor,
        'win-amd64' if os.name == 'nt' else 'linux-x86_64'))[0])
except IndexError:
    pass

```

```

import weakref
import math
import numpy as np
import carla
import ai_knowledge as data
from ai_knowledge import Status

# Monitor is responsible for reading the data from the sensors and telling it to the
knowledge
# TODO: Implement other sensors (lidar and depth sensors mainly)
# TODO: Use carla API to read whether car is at traffic lights and their status, update
it into knowledge
class Monitor(object):
    def __init__(self, knowledge, vehicle):
        self.vehicle = vehicle
        self.knowledge = knowledge
        weak_self = weakref.ref(self)

        self.knowledge.update_data('location', self.vehicle.get_transform().location)
        self.knowledge.update_data('rotation', self.vehicle.get_transform().rotation)

        world = self.vehicle.get_world()
        self.knowledge.update_data('world', world)

        # Lane invasion sensor
        bp = world.get_blueprint_library().find('sensor.other.lane_invasion')
        self.lane_detector = world.spawn_actor(bp, carla.Transform(),
attach_to=self.vehicle)
        self.lane_detector.listen(lambda event: Monitor._on_invasion(weak_self, event))

        # Lidar sensor
        lidar_bp = world.get_blueprint_library().find('sensor.lidar.ray_cast')
        lidar_bp.set_attribute('range', '15')
        lidar_bp.set_attribute('rotation_frequency', '20')
        lidar_bp.set_attribute('channels', '1')
        lidar_bp.set_attribute('points_per_second', '10000')
        lidar_transform = carla.Transform(carla.Location(x=0, z=2.0))
        self.lidar = world.spawn_actor(lidar_bp, lidar_transform, attach_to=self.vehicle)
        self.lidar.listen(lambda lidar_data: Monitor._on_lidar(weak_self, lidar_data))

# Function that is called at time intervals to update ai-state
def update(self, time_elapsed):
    # Update the position of vehicle into knowledge
    self.knowledge.update_data('location', self.vehicle.get_transform().location)
    self.knowledge.update_data('rotation', self.vehicle.get_transform().rotation)

    # Read traffic light state and push into knowledge
    at_red = False
    try:
        if self.vehicle.is_at_traffic_light():
            tl = self.vehicle.get_traffic_light()
            if tl is not None:

```



```

        state = tl.get_state()
        at_red = state in (carla.TrafficLightState.Red,
carla.TrafficLightState.Yellow)
    except Exception:
        pass
    self.knowledge.update_data('at_lights', at_red)

    @staticmethod
    def _on_invasion(weak_self, event):
        self = weak_self()
        if not self:
            return
        self.knowledge.update_data('lane_invasion', event.crossed_lane_markings)

    @staticmethod
    def _on_lidar(weak_self, lidar_data):
        self = weak_self()
        if not self:
            return
        points = []
        for point in lidar_data:
            points.append((point.point.x, point.point.y))
        self.knowledge.update_data('lidar', points)

# Analyser is responsible for parsing all the data that the knowledge has received from
Monitor and turning it into something usable
# TODO: During the update step parse the data inside knowledge into information that
could be used by planner to plan the route
class Analyser(object):
    def __init__(self, knowledge):
        self.knowledge = knowledge
        self.knowledge.set_data_changed_callback(self.on_data_changed)

        # Threat settings for HEALING
        self.threat_active = False
        self.clear_count = 0
        self.CLEAR_THRESHOLD = 4

        self.vehicle = None

    def set_vehicle(self, vehicle):
        self.vehicle = vehicle
        # Default speed. Executor converts to m/s
        self.knowledge.update_data('target_speed', 72)

    # Called by Knowledge.update_data() every time any sensor value changes
    def on_data_changed(self, data_name):
        if data_name == 'lidar':
            self.analyse_lidar()

    # LIDAR-based forward obstacle detection
    def analyse_lidar(self):

```

```

# Use the most recent scan
points = self.knowledge.retrieve_data('lidar') \
    if 'lidar' in self.knowledge.memory else []

# Values for when a point is threatening
FORWARD_MIN = 0.5
FORWARD_MAX = 12.0
LANE_HALF_W = 2.0

threat_points = [
    (x, y) for (x, y) in points
    if FORWARD_MIN < x < FORWARD_MAX and abs(y) < LANE_HALF_W
]

# This part is for checking for threats, it uses cross-referencing with known moving
vehicles
# and it only scans for objects that are moving so that stationary objects are not
detected as threats.
lidar_threat_count = 0
if len(threat_points) >= 3 and self.vehicle is not None:
    ego_loc = self.vehicle.get_transform().location
    for actor in self.vehicle.get_world().get_actors().filter('vehicle.*'):
        if actor.id == self.vehicle.id:
            continue
        vel = actor.get_velocity()
        speed = math.sqrt(vel.x**2 + vel.y**2 + vel.z**2)
        if speed > 0.5 and ego_loc.distance(actor.get_transform().location) < 20.0:
            lidar_threat_count = len(threat_points)
            break

self.knowledge.update_data('lidar_threat_count', lidar_threat_count)

# World-actor proximity and trajectory check
def check_nearby_vehicles(self):
    world = self.vehicle.get_world()
    ego_transform = self.vehicle.get_transform()
    ego_loc = ego_transform.location
    yaw_rad = math.radians(ego_transform.rotation.yaw)

    threat_detected = False
    for actor in world.get_actors().filter('vehicle.*'):
        if actor.id == self.vehicle.id:
            continue

        other_loc = actor.get_transform().location
        world_dist = ego_loc.distance(other_loc)

        # Ignore irrelevant actors
        if world_dist > 25.0:
            continue

        # Transform the other vehicle's position into our local frame
        dx = other_loc.x - ego_loc.x

```

```

dy = other_loc.y - ego_loc.y
local_x = dx * math.cos(yaw_rad) + dy * math.sin(yaw_rad)
local_y = -dx * math.sin(yaw_rad) + dy * math.cos(yaw_rad)

# Sensor settings
if not (-2.0 < local_x <= 20.0 and abs(local_y) < 5.0):
    continue

# Only consider moving vehicles
other_vel = actor.get_velocity()
other_speed = math.sqrt(other_vel.x**2 + other_vel.y**2 + other_vel.z**2)
if other_speed <= 0.5:
    continue

# This is a check made for checking if the other vehicle is moving towards our car
to_ego_x = ego_loc.x - other_loc.x
to_ego_y = ego_loc.y - other_loc.y
to_ego_len = math.sqrt(to_ego_x**2 + to_ego_y**2) + 1e-6
closing = (other_vel.x * to_ego_x + other_vel.y * to_ego_y) / to_ego_len

# Calculation that checks if another car is merging into our lane
lateral_vel = (-other_vel.x * math.sin(yaw_rad) + other_vel.y * math.cos(yaw_rad))
merging_into_lane = (local_y < 0 and lateral_vel > 0.3) or (local_y > 0 and
lateral_vel < -0.3)

# Threat settings
if world_dist < 6.0 or closing > 0.5 or merging_into_lane:
    threat_detected = True
    print(f'[Analyser] Threat: actor={actor.id} '
          f'local_x={local_x:.1f} local_y={local_y:.1f} '
          f'closing={closing:.1f} merging={merging_into_lane}')
    break

# Lidar threat
lidar_threat = self.knowledge.retrieve_data('lidar_threat_count') >= 3 \
    if 'lidar_threat_count' in self.knowledge.memory else False

# Skip threat processing once we have arrived
current_status = self.knowledge.get_status()
if current_status == Status.ARRIVED:
    return

# Brake if threat is detected
if threat_detected or lidar_threat:
    self.threat_active = True
    self.clear_count = 0
    if current_status == Status.DRIVING:
        print('[Analyser] Switching to HEALING')
        self.knowledge.update_status(Status.HEALING)

    control = carla.VehicleControl()
    control.throttle = 0.0
    control.steer = 0.0

```

```

        control.brake          = 1.0
        control.hand_brake    = False
        self.vehicle.apply_control(control)
    else:
        if self.threat_active:
            self.clear_count += 1
            if self.clear_count >= self.CLEAR_THRESHOLD:
                self.threat_active = False
                self.clear_count   = 0
                if current_status == Status.HEALING:
                    print('[Analyser] Threat cleared - resuming DRIVING')
                    self.knowledge.update_status(Status.DRIVING)

#Function that is called at time intervals to update ai-state
def update(self, time_elapsed):
    if self.vehicle is None:
        return

    # Read the traffic light written by monitor
    at_lights = self.knowledge.retrieve_data('at_lights') \
        if 'at_lights' in self.knowledge.memory else False

    if at_lights:
        self.knowledge.update_data('target_speed', 0)
    elif self.knowledge.get_status() != Status.HEALING:
        self.knowledge.update_data('target_speed', 72)

    self.check_nearby_vehicles()

```

ai_test.py

```

#!/usr/bin/env python

# Copyright (c) 2017 Computer Vision Center (CVC) at the Universitat Autònoma de
# Barcelona (UAB).
#
# This work is licensed under the terms of the MIT license.
# For a copy, see <https://opensource.org/licenses/MIT>.

import glob
import math
import os
import sys

try:
    sys.path.append(glob.glob('**/*%d.%d-%s.egg' % (
        sys.version_info.major,
        sys.version_info.minor,
        'win-amd64' if os.name == 'nt' else 'linux-x86_64'))[0])
except IndexError:
    pass

```

```

import carla
import pygame

import random
import time
import argparse

import custom_ai as ai

def get_dist(point1, point2):
    return point1.location.distance(point2)

def get_start_point(world, coord):
    points = world.get_map().get_spawn_points()
    index = 0
    ti = -1
    td = get_dist(points[0], coord)
    for point in points:
        ti += 1
        d = get_dist(point, coord)
        if d < td:
            td = d
            index = ti
    start_point = points[index]
    return world.get_map().get_waypoint(start_point.location)

def main():
    argparser = argparse.ArgumentParser(
        description=__doc__)
    argparser.add_argument(
        '-m', '--milestone-number',
        metavar='M',
        default=1,
        type=int,
        help='Milestone number (default: 1)')
    args = argparser.parse_args()

    actor_list = []

    try:
        client = carla.Client('localhost', 2000)
        client.set_timeout(10.0)
        world = client.get_world()
        world = client.load_world('Town03')
        blueprints = world.get_blueprint_library().filter('vehicle.*')
        blueprints = [x for x in blueprints if int(x.get_attribute('number_of_wheels'))
== 4]

        blueprints = [x for x in blueprints if not x.id.endswith('isetta')]

        def try_spawn_random_vehicle_at(transform, vid=""):
            if vid != "":
                blueprint = None

```

```

        for bp in blueprints:
            print("{} - {}".format(bp.id, bp.id==vid))
            if bp.id == vid:
                blueprint = bp
                break
        if blueprint.has_attribute('color'):
            color =
random.choice(blueprint.get_attribute('color').recommended_values)
            blueprint.set_attribute('color', color)
            blueprint.set_attribute('role_name', 'autopilot')
            vehicle = world.try_spawn_actor(blueprint, transform)
            if vehicle is not None:
                actor_list.append(vehicle)
                print('spawned %r at %s' % (vehicle.type_id, transform.location))
                return vehicle
    else:
        attempts = 20
        while attempts > 0:
            blueprint = random.choice(blueprints)
            if blueprint.has_attribute('color'):
                color =
random.choice(blueprint.get_attribute('color').recommended_values)
                blueprint.set_attribute('color', color)
                blueprint.set_attribute('role_name', 'autopilot')
                vehicle = world.try_spawn_actor(blueprint, transform)
                if vehicle is not None:
                    actor_list.append(vehicle)
                    print('spawned %r at %s' % (vehicle.type_id,
transform.location))
                    return vehicle
        print('WARNING: vehicle not spawned, NONE returned')
        return None

    # Here we added clarity to the test routes as we felt they were confusing and
incorrect.
    # The original ex3 is commented out, and the originally commented ex4 is now ex3
for tests.
    # Defining positions
    ex1 = [ carla.Vector3D(42.5959,-4.3443,1.8431), carla.Vector3D(22,-4,1.8431),
carla.Vector3D(9,-22,1.8431)]
    ex2 = [ carla.Vector3D(42.5959,-4.3443,1.8431), carla.Vector3D(-30,167,1.8431)]
    #ex3 = [ carla.Vector3D(42.5959,-4.3443,1.8431), carla.Vector3D(22,-4,1.8431),
carla.Vector3D(9,-22,1.8431)]
    ex3 = [ carla.Vector3D(42.5959,-4.3443,1.8431), carla.Vector3D(134,-3,1.8431)]

    #kzs2 = carla.Vector3D(-85,-23,1.8431)

    milestones = [ex1, ex2, ex3]
    ms = max(0,min(args.milestone_number-1,len(milestones)-1))
    ex = milestones[ms]
    end = ex[len(ex)-1]
    destination = ex[1]

```

```

# Getting waypoint to spawn
start = get_start_point(world, ex[0])
# Spawning
vehicle = try_spawn_random_vehicle_at(start.transform, "vehicle.nissan.micra")

if vehicle == None:
    return
# Setting autopilot
def route_finished(autopilot):
    pos = autopilot.get_vehicle().get_transform().location
    print ("Vehicle arrived at destination: ", pos)
    if pos.distance(carla.Location(end)) < 5.0:
        print ("Excercise route finished")
        running = False
    else:
        autopilot.set_destination(end)

autopilot = ai.Autopilot(vehicle)
autopilot.set_destination(destination)
autopilot.set_route_finished_callback(route_finished)

if ms == 1:
    print("Spawning malicious actor...")
    spawn = start.get_right_lane()
    mal = try_spawn_random_vehicle_at(spawn.transform, "vehicle.nissan.micra")
    bp = world.get_blueprint_library().find('sensor.other.collusion')
    sensor = world.spawn_actor(bp, carla.Transform(), attach_to=mal)

    def _on_collision(self, event):
        if not self:
            return
        print ('Collision with: ', event.other_actor.type_id)
        if event.other_actor.type_id.split('.')[0] == 'vehicle':
            print ("Test FAILED")
            mal.destroy()
            sensor.destroy()

    sensor.listen(lambda event: _on_collision(mal, event))

    control = carla.VehicleControl()
    control.throttle = 1.0
    control.steer = -0.07
    control.brake = 0.0
    control.hand_brake = False
    mal.apply_control(control)

ctr = 0
running = True

# Here we added our own spectator that follows the car.
# This is better than manual spectating.
spectator = world.get_spectator()

```

```
while running:
    transform = vehicle.get_transform()

    forward = transform.get_forward_vector()

    spectator.set_transform(carla.Transform(
        carla.Location(
            x=transform.location.x - forward.x * 3,
            y=transform.location.y - forward.y * 3,
            z=transform.location.z + 4
        ),
        carla.Rotation(pitch=-15, yaw=transform.rotation.yaw)
    ))

    status = autopilot.update()
    if status == None:
        ctr += 1
        if ctr > 3:
            running = False
    else:
        ctr = 0

    time.sleep(0.001)

finally:

    print('destroying actors')
    for actor in actor_list:
        actor.destroy()
    print('done.')

    pygame.quit()

if __name__ == '__main__':

    main()
```